

7주차 과제D

사람 백소정

요약

#	작업	파일	상태
1	정적/동적/네트워크 결과를 합쳐 종합 위험도 산정	<code>src/risk_aggregator.py</code> (신규)	✅ 구현 + 검증
2	main.py의 빈 <code>risk_score</code> 플레이스홀더를 실제 계산으로 배선	<code>src/main.py</code>	✅
3	대시보드 통일 팔레트 (위험 등급색 + 모듈색) 추가	<code>common.py</code>	✅
4	종합 위험도 패널 (게이지 + 등급 배지 + 모듈 기여도)	<code>views/risk_view.py</code>	✅ 구현 + 검증
5	네트워크 뷰 (의심 강조 + 화이트리스트 대비 + DNS/TLS 표)	<code>views/network_view.py</code>	✅ 구현 + 검증
6	누락된 <code>streamlit</code> 의존성 추가 + <code>jsonschema</code> 버전 조정	<code>requirements.txt</code>	⚠️ B 재확인 필요

브랜치: `feature/risk-aggregator` (1~2번은 커밋/PR 완료, 3~6번은 아직 미커밋 상태에서 이 기록 작성).

1. 종합 위험도 스코어링 (`src/risk_aggregator.py`)

세 모듈이 이미 뽑아둔 신호를 각각 0.0~1.0 하위점수(sub-score)로 환산한 뒤, 모듈 가중치로 **가중평균**해서 종합 `total / level / breakdown` 을 낸다.

- **정적**: `risk_scorer` 가 이미 낸 `risk_score` (0~1)를 그대로 인용 (중복 계산 안 함)
- **동적**: `plaintext_candidates` 수 + `hook_type` 별 이벤트 수를 가중 합산 후 정규화
- **네트워크**: 이미 판별된 `suspicious` 도메인/IP 개수를 가중 합산 후 정규화

핵심 정책:

- 실패/타임아웃 모듈은 **빼고** 남은 모듈끼리 가중치를 **재정규화**한다.
- 세 모듈이 다 못 쓰면 `total=None`, `level="unknown"` — 실패를 0.0("안전")으로 오해하지 않게 함.
- 가중치(static 0.4 / dynamic 0.3 / network 0.3), 상한(CAP), 등급 임계값 (low<0.34 / medium<0.67 / high)은 전부 상단 상수. **실샘플 검증 후 재조정 필요한 초기 추정값**이다.

`main.py` 는 `run_pipeline()` 끝에서 `aggregate_risk(modules)` 를 호출해 `risk_score` 필드를 채우고, CLI 출력에도 종합 점수를 찍는다.

검증: 4개 시나리오(악성 가정 → high, 정상 가정 → low, 정적만 살아있음 → medium+가중치 재정규화, 전부 실패 → unknown) 로직 확인 완료.

2. 대시보드 통일 팔레트 (`common.py`)

4명 산출물이 한 세트처럼 보이도록 색 표를 공용으로 뺐다 (dataviz 스킬의 검증된 status/카테고리 팔레트 사용).

- `RISK_LEVEL_COLORS/ICONS/LABELS_KO` : low/medium/high/unknown → 낮음/주의/위험/판정불가, 색은 초록/노랑/빨강/회색(status 팔레트)

- `MODULE_COLORS` : static/dynamic/network = 파랑/초록/마젠타 (CVD 검증 통과)
- `risk_level_ko()`, `risk_level_color()` 헬퍼

B가 정적 뷰에서 권한을 강조할 때 `RISK_LEVEL_COLORS`의 high/medium/low를 그대로 쓰면 D의 위험도 게이지와 색이 통일된다 (파일 주석에도 명시).

3. 종합 위험도 패널 (`views/risk_view.py`)

- 히어로 점수(0~100) + 등급 배지 (색 + 아이콘 + 한글, 색 단독 사용 안 함)
- 게이지: 저/중/고 구간 밴드(status 색) + 현재 점수 눈금/점
- 모듈별 기여도: 가로 막대(모듈색) + 직접 라벨 + 표 병기 ($\text{기여도} = \text{하위점수} \times \text{재정규화 가중치}$, 막대 합 = 종합 점수)
- `aggregate_risk()` 의 실제 출력 형태 그대로 소비한다. 표시할 때만 $\times 100$, 한글 등급으로 변환.
- `total=None` 이면 실제 출력과 같은 형태의 더미 데이터를 그리되 "예시 데이터" 배너를 띄워 실측과 구분 → 실제 계산 붙으면 뷰 코드 그대로 살아남.

4. 네트워크 뷰 (`views/network_view.py`)

- 의심 도메인/IP를 맨 위에 빨강(critical)으로 강조 (배너 + 사유 표)
- 화이트리스트 대비 결과: 통과 vs 의심 비율 막대 + metric 4종 (관측 도메인 / 통과 / 의심 / DNS·TLS 레코드 수)
- `dns_queries` / `tls_sni` 표 — 의심 항목은 표시
- pcap pull 실패 등으로 필드가 비어도 `safe_get` 으로 방어

5. 의존성 변경 (`requirements.txt`) — ! 검토 필요

- `streamlit==1.61.1` 추가: 대시보드(app.py 등)가 이미 있는데 requirements에 streamlit이 아예 빠져 있었음. altair/pandas는 streamlit이 함께 설치.
- `jsonschema 3.2.0 → 4.26.0`: streamlit의 altair가 더 높은 버전을 요구해 충돌.
B의 검증 코드는 `Draft7Validator` (단일 스키마)만 쓰고 `$ref / RefResolver` 를 안 써서 4.x에서도 API 동일하게 동작하는 것 확인함. 다만 B가 자기 테스트 44개를 4.26.0에서 한 번 재확인하는 게 안전.

검증 방법 / 도구

- 팔레트: dataviz 스킴 검증기(`validate_palette.js`)로 모듈 3색을 라이트/다크 모두 PASS 확인 (라이트 magenta 대비 WARN → 막대 직접 라벨 + 표 병기로 relief 충족).
- 뷰 렌더링: streamlit `AppTest` 로 headless 렌더 — 4개 시나리오 (실측 / 더미 / 부분실패 / 의심없음) 전부 예외 없이 렌더 확인.
- `py_compile` 통과, deprecated API(`use_container_width`) 제거.

아직 안 한 것 / 남은 것

- 라이브 화면 스크린샷: headless 렌더는 통과했지만 `streamlit run app.py` 로 실제 브라우저에서 레이아웃(라벨 겹침 등)을 눈으로 확인하는 건 아직 안 함.

- 위험도 가중치/임계값 실샘플 검증: 정상 앱 vs Anubis 계열 샘플로 돌려보고 가중치·등급 경계를 팀이 함께 조정해야 함 (현재는 초기 추정값).
- B의 jsonschema 4.26.0 회귀 확인 (위 5번).

▼ 설명

오늘 requirements.txt를 건드리면서 생긴 jsonschema 버전 변경 때문에 붙인 메모입니다.

무슨 일이 있었나

- 대시보드용 streamlit을 requirements에 추가했는데, streamlit이 달려오는 altair가 최신 jsonschema를 요구합니다.
- 그래서 jsonschema가 기존 3.2.0 → 4.26.0으로 올라갔습니다.

왜 B랑 관련이 있나

- B가 정적 분석 어댑터 테스트에서 jsonschema로 스키마 검증을 합니다 (tests/test_static_adapter.py — 44개 테스트, Draft7Validator로 static_report.schema.json 통과 여부 확인).
- 즉 B의 테스트가 jsonschema 라이브러리에 직접 의존하는데, 그 라이브러리 버전을 제가 바꿔버린 상황입니다.

위험한가? (제가 이미 확인한 것)

대체로 안전합니다:

- jsonschema 4.x에서 큰 변경은 RefResolver/\$ref 처리 방식인데, B 코드는 Draft7Validator로 단일 스키마 파일 하나만 검증하고 \$ref나 RefResolver를 안 씁니다.
- 그 API(Draft7Validator, validate)는 3.x → 4.x에서 동일하게 동작하는 것을 제가 확인했습니다.
- Draft-07 검증 규칙 자체도 그대로입니다.

그럼 "B 재확인 필요"는 왜?

제가 확인한 건 API 호환성까지고, B의 44개 테스트를 실제로 4.26.0에서 돌려본 건 아닙니다 (이 PC엔 androguard가 없어서 그 테스트 체인이 여기서 안 돛). 그래서:

■ B가 자기 환경에서 pytest를 한 번 돌려 44개가 여전히 통과하는지 확인하면 끝입니다.

- 3~6번 변경의 커밋/PR 방식 결정 (위험도 PR에 합칠지, 대시보드 별도 브랜치로 뺄지).